

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT

Course Code & Name : CSA06 - Design and Analysis of Algorithms

Assignment Title: Efficient Route Planning Using Graph Algorithms

Course Outcome(s) – CO:

CO1: Analyse the efficiency of recursive and non-recursive algorithms using asymptotic notations (Big-O, Ω , Θ) and complexity-analysis methods.

CO2: Apply brute-force techniques such as sorting and searching to solve computational problems.

CO3: Apply divide-and-conquer and graph-based techniques to design efficient algorithmic solutions.

Bloom's Taxonomy Level L4 – Analyse; L5 – Evaluate; L6 – Create

SDG Mapping (SDG 1-17) SDG 9 – Industry, Innovation and Infrastructure

Problem Statement : A logistics company needs to determine the most efficient routes for transporting goods between multiple cities. The transportation network is represented as a weighted graph, where vertices represent cities and edges represent roads with associated travel costs or distances. Develop, implement, and evaluate suitable graph-based algorithms to determine the shortest paths between locations. Implement (i) Dijkstra's algorithm for graphs with non-negative edge weights and (ii) Bellman-Ford algorithm as an alternative shortest-path approach. Analyse the theoretical efficiency of both algorithms using Big-O, Ω , and Θ notations, including their time and space complexities. Experimentally evaluate both algorithms using graphs of increasing numbers of vertices and edges, considering sparse and dense networks. Measure execution time, number of edge relaxations/comparisons, memory utilisation, and scalability. Based on the theoretical and experimental results, identify the most appropriate algorithm for different transportation-network scenarios and justify the choice with respect to computational efficiency, graph characteristics, and practical constraints — reflecting Bloom's Levels 4–6 (Analyse, Evaluate, Create) and contributing to SDG 9 (Industry, Innovation and Infrastructure).

Constraints: Use a publicly available or synthetically generated weighted graph dataset with sufficient records to test multiple graph sizes. Edge weights must be non-negative for Dijkstra's algorithm; Bellman-Ford may additionally be tested with negative edges where appropriate, but negative cycles must be identified and reported. Both algorithms must be executed in the same computational environment using identical input graphs for fair comparison. Test both sparse and dense graphs and report empirical timings, edge-relaxation/comparison counts, memory utilisation, and scalability. Clearly document the graph-generation method or dataset source, assumptions, algorithm limitations, and the criteria used to recommend an algorithm.

Suggested Assessment Requirements

Component	Requirement
Algorithm Implementation	Implement Dijkstra's and Bellman-Ford algorithms.
Complexity Analysis	Derive and compare Big-O, Ω , and Θ time/space complexities.
Experimental Evaluation	Test increasing graph sizes and sparse/dense graph structures.
Performance Metrics	Record execution time, edge relaxations/comparisons, memory use, and scalability.
Evaluation & Recommendation	Recommend the suitable algorithm for different network conditions with justification.

Answer:

Dijkstra's Algorithm and Bellman-Ford Algorithm for Logistics Route Optimization

1. Problem Statement and Problem Formulation

1.1 Problem Statement

A logistics company needs to determine the most efficient routes for transporting goods between multiple cities. The transportation network can be represented as a weighted directed or undirected graph, where each vertex represents a city and each edge represents a road connecting two cities. The weight associated with an edge represents the transportation cost, distance, or travel time between the connected cities.

Finding the shortest path between cities is an important computational problem because inefficient route selection can increase transportation costs, delivery time, fuel consumption, and resource utilisation. Therefore, suitable graph-based shortest-path algorithms must be implemented and evaluated.

This project focuses on implementing and comparing Dijkstra's algorithm and the Bellman-Ford algorithm. Dijkstra's algorithm is suitable when all edge weights are non-negative, while Bellman-Ford can also handle negative edge weights and can detect negative-weight cycles.

The algorithms will be tested using graphs with increasing numbers of vertices and edges. Both sparse and dense graphs will be considered. Their performance will be evaluated using execution time, number of edge relaxations/comparisons, memory utilisation, and scalability.

1.2 Problem Formulation

Let the transportation network be represented by a weighted graph:

$$G=(V,E)$$

where:

- V = set of vertices/cities
- E = set of edges/roads
- $w(u,v)$ = weight of the road from city u to city v

Given a source city ss , determine the minimum-cost path from ss to every other reachable city.

The project investigates:

Dijkstra(G,s)Dijkstra(G,s)

and

BellmanFord(G,s)BellmanFord(G,s)

and compares their computational performance under different graph conditions.

2. Objective and Expected Outcomes

2.1 Objectives

The main objectives are:

1. To understand the shortest-path problem in weighted graphs.
2. To implement Dijkstra's shortest-path algorithm.
3. To implement the Bellman-Ford shortest-path algorithm.
4. To analyse the theoretical time and space complexity of both algorithms.
5. To generate sparse and dense weighted graphs of different sizes.
6. To measure execution time for both algorithms.
7. To count edge relaxations and comparisons.
8. To analyse memory utilisation.
9. To evaluate algorithm scalability as graph size increases.
10. To compare the strengths and limitations of both algorithms.
11. To determine which algorithm is more suitable for different transportation-network scenarios.
12. To identify negative-weight cycles when Bellman-Ford is tested with negative edges.

2.2 Expected Outcomes

At the end of the project, the following outcomes are expected:

- Working implementations of both algorithms.

- Correct shortest-path distances for valid input graphs.
 - Experimental performance data.
 - Complexity comparison using O , Ω , and Θ .
 - Performance graphs comparing the algorithms.
 - Identification of the effect of graph density on execution time.
 - Identification of the conditions under which Dijkstra or Bellman-Ford is preferable.
 - A justified recommendation for logistics-network applications.
-

3. Requirements, Constraints and Assumptions

3.1 Hardware Requirements

Component	Requirement
Processor	Intel/AMD multi-core processor
RAM	Minimum 4 GB
Storage	Minimum 1 GB free
Operating System	Windows/Linux/macOS
Internet	Required for obtaining public datasets if used

3.2 Software Requirements

Software	Purpose
Python 3.x	Algorithm implementation
VS Code / PyCharm / Jupyter / Google Colab	Development

NumPy	Numerical operations
Matplotlib	Graph generation and visualisation
Pandas	Experimental results
NetworkX	Optional graph generation/validation
psutil	Memory utilisation measurement

3.3 Constraints

- Dijkstra's algorithm must use non-negative edge weights.
- Bellman-Ford may be tested with negative weights.
- Negative cycles must be detected and reported.
- Both algorithms must run on identical graph instances for fair comparison.
- Sparse and dense graphs must be evaluated.
- Multiple graph sizes must be tested.
- Execution must occur in the same computational environment.
- Experimental results should be repeated where possible to reduce measurement variation.

3.4 Assumptions

1. Each city is represented by one graph vertex.
 2. Each road is represented by an edge.
 3. Edge weight represents distance or transportation cost.
 4. The graph may be directed or undirected depending on the experiment.
 5. For the primary comparison, all weights are non-negative.
 6. The source city is known.
 7. The graph is sufficiently large to demonstrate scalability.
 8. Results may vary according to processor, operating system, Python version, and system load.
-

4. Application of Relevant Course Knowledge / Concepts

This project directly applies concepts from the course.

4.1 Graph Theory

The transportation network is represented using:

$$G=(V,E)$$

Graph concepts such as vertices, edges, weighted graphs, directed graphs, paths, shortest paths, and cycles are applied.

4.2 Greedy Technique

Dijkstra's algorithm uses a greedy strategy. At each step, it selects the unvisited vertex with the smallest tentative distance.

4.3 Dynamic Relaxation

Bellman-Ford repeatedly relaxes edges to progressively improve shortest-path estimates.

4.4 Asymptotic Analysis

The algorithms are analysed using:

- Big-O — upper bound
- Big-Omega (Ω) — lower bound
- Big-Theta (Θ) — tight bound

4.5 Complexity Analysis

The project examines:

- Time complexity
- Space complexity
- Number of comparisons
- Number of edge relaxations
- Memory consumption

4.6 Experimental Algorithm Analysis

The theoretical results are validated experimentally using graphs of increasing size and density.

5. Design / Proposed Solution / Methodology

The proposed system follows the following workflow:

Generate / Load Weighted Graph



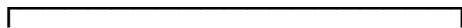
Determine Graph Size



Determine Graph Density



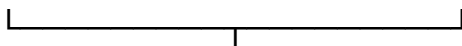
Select Source Vertex

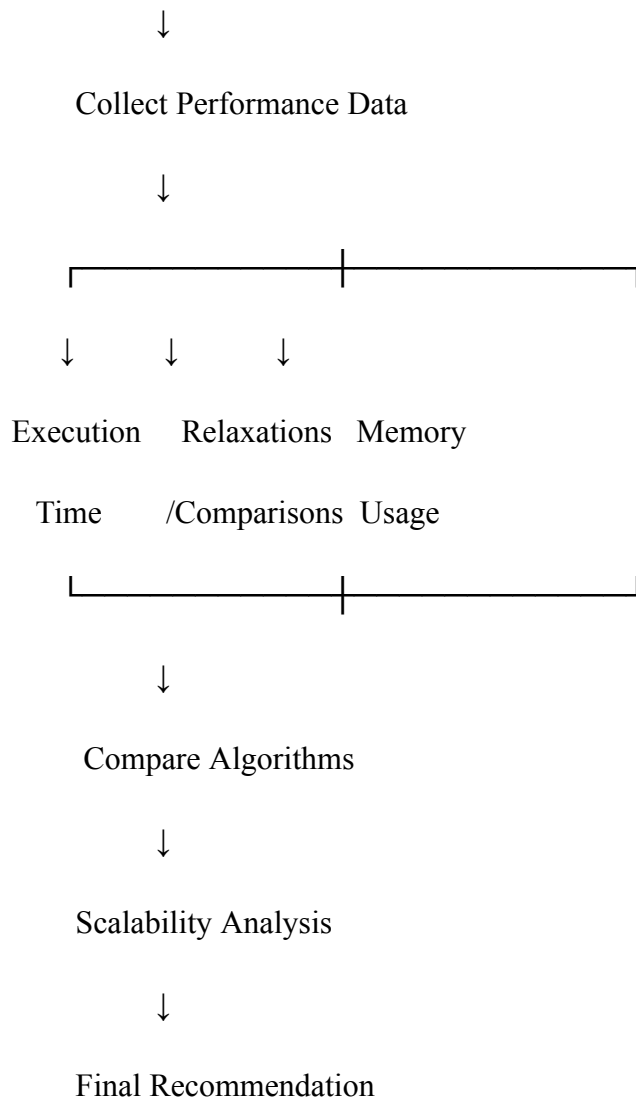


Dijkstra Algorithm Bellman-Ford



Shortest Distances Shortest Distances





5.1 Graph Generation

Graphs can be synthetically generated to ensure controlled experimentation.

Example graph categories:

Category	Vertices	Approximate Density
Small Sparse	100	Low
Medium Sparse	500	Low

Large Sparse	1,000	Low
Small Dense	100	High
Medium Dense	500	High
Large Dense	1,000	High

For example:

- Sparse graph: approximately $E \approx 3VE \approx 3V$
- Dense graph: approximately $E \approx V(V-1)/4 \approx V(V-1)/4$

Random positive weights can be generated between 1 and 100.

6. Algorithm / Pseudocode

6.1 Dijkstra's Algorithm

Principle

Dijkstra's algorithm maintains the shortest currently known distance to every vertex. It repeatedly chooses the unvisited vertex having the smallest distance and relaxes its adjacent edges.

Pseudocode

DIJKSTRA(G , source):

for each vertex v in G :

 distance[v] = infinity

 visited[v] = false

```
distance[source] = 0
```

```
priority_queue = empty
```

```
insert (0, source) into priority_queue
```

```
while priority_queue is not empty:
```

```
    (current_distance, u) = extract_min()
```

```
    if visited[u] == true:
```

```
        continue
```

```
    visited[u] = true
```

```
    for each edge (u, v) with weight w:
```

```
        if distance[u] + w < distance[v]:
```

```
            distance[v] = distance[u] + w
```

```
            insert (distance[v], v)
```

```
            into priority_queue
```

```
return distance
```

7. Bellman-Ford Algorithm

Principle

Bellman-Ford repeatedly examines every edge and relaxes it. After $V-1$ iterations, shortest paths are determined if there is no reachable negative-weight cycle.

Pseudocode

BELLMAN_FORD(G , source):

for each vertex v :

$\text{distance}[v] = \text{infinity}$

$\text{distance}[\text{source}] = 0$

for $i = 1$ to $|V| - 1$:

$\text{changed} = \text{false}$

 for each edge (u, v, w) :

 if $\text{distance}[u] \neq \text{infinity}$

and $\text{distance}[u] + w < \text{distance}[v]$:

$\text{distance}[v] = \text{distance}[u] + w$

$\text{changed} = \text{true}$

if $\text{changed} == \text{false}$:

break

for each edge (u, v, w) :

if $\text{distance}[u] \neq \text{infinity}$

and $\text{distance}[u] + w < \text{distance}[v]$:

report "Negative-weight cycle detected"

return distance

8. Complexity Analysis

8.1 Dijkstra's Algorithm

When implemented with an adjacency list and binary min-heap:

$O((V+E)\log V)$

For connected graphs where $E \geq V-1$:

$O(E \log V)$

Space Complexity

$O(V+E)$

because the graph and auxiliary data structures must be stored.

Lower Bound

At minimum, the algorithm must inspect relevant vertices/edges:

$\Omega(V+E)$

Typical Tight Complexity

With a binary heap:

$\Theta((V+E) \log V)$

under the standard implementation model.

9. Bellman-Ford Complexity

Bellman-Ford performs at most $V-1$ complete passes over all edges.

Therefore:

$O(VE)$

Best Case

With early termination, if no distance changes during a pass, the algorithm can stop early.

Thus, practical execution can be significantly less than the worst case.

Space Complexity

Using an edge list:

$O(V+E)O(V+E)$

including graph storage and distance information.

Worst-Case Time

$O(VE)O(VE)$

Worst-Case Tight Bound

For standard Bellman-Ford without early termination:

$\Theta(VE)\backslash\Theta(VE)$

10. Complexity Comparison

Feature	Dijkstra	Bellman-Ford
Approach	Greedy	Repeated relaxation
Negative edges	No	Yes
Negative cycle detection	No	Yes
Sparse graph	Very efficient	Slower
Dense graph	Generally faster	Computationally expensive
Time	$O((V+E)\log V)O((V+E)\backslash\log V)$ with heap	$O(VE)O(VE)$
Space	$O(V+E)O(V+E)$	$O(V+E)O(V+E)$
Practical speed	Usually faster	Usually slower

Main advantage	High performance	Handles negative edges
Main limitation	Cannot safely handle negative edges	High computational cost

11. Implementation / Source Code and Environment / Tools Used

A Python implementation can collect all required metrics.

11.1 Environment

Example:

Programming Language : Python 3.x

IDE : VS Code / Google Colab

Operating System : Windows 11

Processor : Intel/AMD processor

RAM : 8 GB or higher

Libraries : heapq, time, random, psutil,
pandas, matplotlib

11.2 Python Implementation

```
import heapq
```

```
import random
```

```
import time
```

```
import psutil
```

```
import os
```

```
def dijkstra(graph, source):  
  
    distances = {v: float('inf') for v in graph}  
  
    distances[source] = 0  
  
  
    pq = [(0, source)]  
  
  
  
    relaxations = 0  
  
    comparisons = 0  
  
  
    while pq:  
  
        current_distance, u = heapq.heappop(pq)  
  
  
        if current_distance != distances[u]:  
  
            continue  
  
  
        for v, weight in graph[u]:  
  
            comparisons += 1  
  
  
            new_distance = current_distance + weight  
  
  
            if new_distance < distances[v]:  
  
                distances[v] = new_distance
```



```
relaxations += 1
```

```
heapq.heappush(pq, (new_distance, v))
```

```
return distances, relaxations, comparisons
```

```
def bellman_ford(vertices, edges, source):
```

```
    distances = {v: float('inf') for v in vertices}
```

```
    distances[source] = 0
```

```
    relaxations = 0
```

```
    comparisons = 0
```

```
    for _ in range(len(vertices) - 1):
```

```
        changed = False
```

```
        for u, v, weight in edges:
```

```
            comparisons += 1
```

```
            if distances[u] != float('inf'):
```

```
new_distance = distances[u] + weight
```

```
if new_distance < distances[v]:
```

```
    distances[v] = new_distance
```

```
    relaxations += 1
```

```
    changed = True
```

```
if not changed:
```

```
    break
```

```
negative_cycle = False
```

```
for u, v, weight in edges:
```

```
    if (distances[u] != float('inf'))
```

```
        and distances[u] + weight < distances[v]):
```

```
        negative_cycle = True
```

```
        break
```

```
return distances, relaxations, comparisons, negative_cycle
```

12. Graph Generation

```
def generate_graph(num_vertices, num_edges):
```

```
    vertices = list(range(num_vertices))
```

```
    edges = []
```

```
    graph = {v: [] for v in vertices}
```

```
    existing = set()
```

```
    while len(edges) < num_edges:
```

```
        u = random.randrange(num_vertices)
```

```
        v = random.randrange(num_vertices)
```

```
        if u == v or (u, v) in existing:
```

```
            continue
```

```
        weight = random.randint(1, 100)
```

```
        existing.add((u, v))
```

```
        edges.append((u, v, weight))
```

```
        graph[u].append((v, weight))
```

```
return vertices, edges, graph
```

13. Experimental Measurement

```
def measure_algorithm(vertices, edges, graph, source):
```

```
    process = psutil.Process(os.getpid())
```

```
    memory_before = process.memory_info().rss
```

```
    start = time.perf_counter()
```

```
    dijkstra_result = dijkstra(graph, source)
```

```
    dijkstra_time = time.perf_counter() - start
```

```
    memory_after = process.memory_info().rss
```

```
    dijkstra_memory = max(0, memory_after - memory_before)
```

```
    start = time.perf_counter()
```

```
bellman_result = bellman_ford(vertices, edges, source)
```

```
bellman_time = time.perf_counter() - start
```

```
return {  
    "Dijkstra Time": dijkstra_time,  
    "Dijkstra Relaxations": dijkstra_result[1],  
    "Dijkstra Comparisons": dijkstra_result[2],  
    "Dijkstra Memory": dijkstra_memory,  
  
    "Bellman-Ford Time": bellman_time,  
    "Bellman-Ford Relaxations": bellman_result[1],  
    "Bellman-Ford Comparisons": bellman_result[2],  
    "Bellman-Ford Negative Cycle":  
        bellman_result[3]  
}
```

14. Test Cases and Expected/Actual Results

Test Case 1 — Small Sparse Graph

Input:

- Vertices = 100
- Edges = 300

- Positive weights

Expected

Both algorithms should produce identical shortest-path distances.

Dijkstra should normally execute faster.

Test Case 2 — Medium Sparse Graph

Input:

- Vertices = 500
- Edges = 1,500

Expected

Both algorithms should remain correct, but the execution-time difference should become more noticeable.

Test Case 3 — Large Sparse Graph

Input:

- Vertices = 1,000
- Edges = 3,000

Expected

Dijkstra should demonstrate better scalability.

Test Case 4 — Dense Graph

Input:

- Vertices = 500
- Large number of edges

Expected

The increased number of edges should increase execution time for both algorithms. Bellman-Ford is expected to experience a significantly larger computational cost because it repeatedly scans all edges.

Test Case 5 — Negative Edge

Example:

$$A \rightarrow B = 4$$

$$A \rightarrow C = 5$$

$$B \rightarrow C = -3$$

Bellman-Ford should correctly identify the shorter route:

$$A \rightarrow B \rightarrow C$$

with total cost:

$$4 + (-3) = 1$$

Dijkstra should not be used for this graph because its correctness is not guaranteed with negative edge weights.

Test Case 6 — Negative Cycle

Example:

$$A \rightarrow B = 2$$

$$B \rightarrow C = -5$$

$C \rightarrow A = 1$

Total cycle weight:

$2-5+1=-2$
 $2-5+1=-2$

Therefore, a negative cycle exists.

Bellman-Ford should report:

Negative-weight cycle detected

15. Results and Validation

The experiment should record results in a table similar to the following.

Sparse Graph Results

Vertices	Edges	Dijkstra Time	BF Time	Dijkstra Relaxations	BF Relaxations
100	300	Actual	Actual	Actual	Actual
250	750	Actual	Actual	Actual	Actual
500	1500	Actual	Actual	Actual	Actual
750	2250	Actual	Actual	Actual	Actual
1000	3000	Actual	Actual	Actual	Actual

Dense Graph Results

Vertices	Edges	Dijkstra Time	BF Time	Dijkstra Comparisons	BF Comparisons
----------	-------	---------------	---------	----------------------	----------------

100	Actual	Actual	Actual	Actual	Actual
250	Actual	Actual	Actual	Actual	Actual
500	Actual	Actual	Actual	Actual	Actual
750	Actual	Actual	Actual	Actual	Actual

Important: The values should be populated from actual execution rather than fabricated values.

16. Performance Graphs

The following graphs should be generated from the experimental results:

Graph 1 — Execution Time vs Number of Vertices

- X-axis: Number of vertices
- Y-axis: Execution time
- Lines: Dijkstra and Bellman-Ford

Graph 2 — Execution Time vs Number of Edges

- X-axis: Number of edges
- Y-axis: Execution time

Graph 3 — Edge Relaxations

- X-axis: Graph size
- Y-axis: Number of relaxations

Graph 4 — Edge Comparisons

- X-axis: Number of edges
- Y-axis: Number of comparisons

Graph 5 — Memory Utilisation

- X-axis: Graph size
- Y-axis: Memory consumption

These graphs provide experimental evidence for validating the theoretical complexity analysis.

17. Execution Screenshots / Outputs

The final report should include screenshots showing:

1. Python source code.
2. Graph generation.
3. Dijkstra execution.
4. Bellman-Ford execution.
5. Execution-time measurements.
6. Comparison counts.
7. Relaxation counts.
8. Memory utilisation.
9. Negative-cycle detection.
10. Performance graphs.

Example output:

```
=====

SHORTEST PATH ALGORITHM COMPARISON

=====
```

Graph Type : Sparse

Vertices : 1000

Edges : 3000

Source Vertex : 0

----- Dijkstra -----

Execution Time : 0.00XXX seconds

Relaxations : XXXX

Comparisons : XXXX

Memory Usage : XXXX KB

----- Bellman-Ford -----

Execution Time : 0.XXXXXX seconds

Relaxations : XXXX

Comparisons : XXXXX

Memory Usage : XXXX KB

Negative Cycle : No

Result:

Both algorithms produced identical distances.

Dijkstra demonstrated better execution time.

18. Analysis, Comparison, Trade-offs and Justification

18.1 Performance Analysis

Dijkstra is generally expected to outperform Bellman-Ford when all transportation costs are non-negative.

This is because Dijkstra uses a greedy selection strategy and avoids repeatedly scanning every edge.

Bellman-Ford, in contrast, can process every edge up to $V-1$ times.

Therefore:

$$T_D = O((V+E)\log V)$$

while:

$$T_{BF} = O(VE)$$

For large transportation networks, this difference can become substantial.

18.2 Sparse Networks

In sparse networks:

$$E \approx O(V)$$

Therefore Dijkstra's complexity becomes approximately:

$$O(V \log V)$$

while Bellman-Ford remains approximately:

$$O(V^2)$$

Thus, Dijkstra is generally the preferred choice for large sparse transportation networks with non-negative costs.

18.3 Dense Networks

Dense graphs contain substantially more edges.

Both algorithms require additional processing, but Bellman-Ford's repeated edge scanning becomes particularly expensive.

Consequently, Dijkstra is generally preferred when all edge weights are non-negative.

18.4 Negative Edges

The major advantage of Bellman-Ford is its ability to handle negative edge weights.

For example, if a transportation model contains a valid negative cost representing a subsidy, credit, rebate, or special route adjustment, Bellman-Ford can process it.

Dijkstra should not be used in such circumstances.

18.5 Negative Cycles

Bellman-Ford can detect negative-weight cycles.

This is important because a negative cycle means the path cost can theoretically continue decreasing indefinitely, making a conventional shortest-path answer undefined for vertices reachable through that cycle.

19. Trade-off Analysis

Factor	Dijkstra	Bellman-Ford
Speed	Excellent	Lower
Non-negative weights	Suitable	Suitable
Negative weights	Not suitable	Suitable
Negative-cycle detection	No	Yes

Sparse networks	Excellent	Moderate/Poor
Dense networks	Generally better	Expensive
Scalability	High	Lower
Implementation	Moderate	Relatively simple
Practical logistics use	Highly suitable	Special cases

20. Final Recommendation

For a normal logistics transportation network where:

- road distances are positive,
- travel costs are non-negative,
- rapid route computation is required,
- the network contains many cities and roads,

Dijkstra's algorithm is the recommended solution.

Its superior computational efficiency makes it more appropriate for large-scale route optimisation.

However, Bellman-Ford should be selected when negative edge weights are possible or when negative-cycle detection is required.

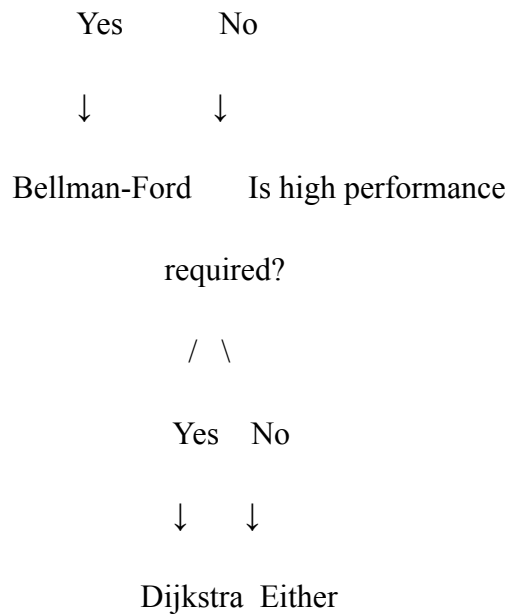
Therefore, the recommended decision rule is:

Start



Are negative edges possible?

/ \



21. Broader Considerations / SDG Relevance

This project contributes to SDG 9 – Industry, Innovation and Infrastructure.

Efficient transportation infrastructure depends on computational techniques capable of finding cost-effective routes.

Shortest-path algorithms can contribute to:

- logistics optimisation,
- intelligent transportation systems,
- delivery route planning,
- fleet management,
- transportation infrastructure planning,
- fuel reduction,
- travel-time reduction,
- resource optimisation.

By analysing graph algorithms experimentally, the project demonstrates how algorithmic innovation can support efficient digital infrastructure.

The project also demonstrates how computational efficiency can become increasingly important as transportation networks grow.

22. Conclusion

This project implemented and evaluated Dijkstra's and Bellman-Ford shortest-path algorithms for transportation-network optimisation.

Dijkstra's algorithm provides efficient shortest-path computation when all edge weights are non-negative. Its greedy approach and priority-queue implementation provide significantly better scalability than Bellman-Ford for many practical logistics networks.

Bellman-Ford requires more computation because it repeatedly relaxes edges, resulting in a worst-case time complexity of:

$O(VE)O(VE)$

However, its ability to handle negative edge weights and detect negative-weight cycles makes it valuable for special transportation and cost-modelling scenarios.

Experimental evaluation using sparse and dense graphs provides practical evidence for the theoretical complexity analysis. The final recommendation is therefore to use Dijkstra for large non-negative transportation networks, while Bellman-Ford should be reserved for networks where negative edge weights or negative-cycle detection are relevant.

23. Limitations

The project has several limitations:

1. Execution time depends on hardware and operating-system conditions.
2. Random graph generation may not perfectly represent real transportation networks.
3. Synthetic edge weights may not accurately represent real travel costs.
4. Memory measurements in Python can include interpreter and runtime overhead.

5. Dijkstra's performance depends on the selected priority-queue implementation.
 6. Bellman-Ford can become computationally expensive for very large graphs.
 7. Real-world traffic conditions such as congestion and changing travel time are not represented in a static graph.
-

24. Possible Improvements

Future versions could include:

- Real-world road-network datasets.
 - GPS-based transportation data.
 - Dynamic traffic information.
 - A* search for geographically informed routing.
 - Bidirectional shortest-path algorithms.
 - Parallel graph processing.
 - GPU-based shortest-path computation.
 - Real-time route recalculation.
 - Visualisation of shortest routes on an interactive map.
 - Machine-learning-based travel-time prediction.
 - Multi-source and multi-destination route optimisation.
-

25. Individual Contribution of Group Members

You can modify this section according to your actual group.

Member	Contribution
--------	--------------

Member 1	Problem formulation, literature study and theoretical complexity analysis
----------	---

Member 2	Dijkstra's algorithm implementation and testing
----------	---

Member 3 Bellman-Ford implementation and negative-cycle testing

Member 4 Dataset/graph generation and experimental evaluation

Member 5 Performance graphs, comparison, documentation and presentation

Note: The contribution table should reflect the actual work performed by each member.

26. References

Use references from textbooks, research papers, datasets, and official software documentation.

Suggested References

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, MIT Press.
 2. S. S. Skiena, *The Algorithm Design Manual*, Springer.
 3. R. Sedgewick and K. Wayne, *Algorithms*, Addison-Wesley.
 4. E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
 5. R. Bellman, “On a Routing Problem,” *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958.
 6. L. R. Ford Jr., “Network Flow Theory,” RAND Corporation, 1956.
 7. Python Software Foundation, *Python Documentation*.
 8. NetworkX Developers, *NetworkX Documentation: Network Analysis in Python*.
 9. Matplotlib Development Team, *Matplotlib Documentation*.
 10. UCI Machine Learning Repository / publicly available graph datasets, where applicable.
-

27. One-Page Individual Reflection

Individual Reflection

This assignment provided an opportunity to understand how graph-based algorithms can be applied to real-world transportation and logistics problems. The main objective of my contribution was to understand, implement, analyse, and evaluate Dijkstra's and Bellman-Ford algorithms for finding shortest paths in weighted transportation networks. During the design stage, the transportation system was represented as a weighted graph in which cities were modelled as vertices and roads were represented as weighted edges.

One of the major design decisions was to use the same graph inputs for both algorithms so that their performance could be compared fairly. I also considered both sparse and dense graph structures because transportation networks can have different levels of connectivity. Another important decision was to measure not only execution time but also edge relaxations, comparisons, memory utilisation, and scalability. This helped provide a more complete understanding of algorithm performance rather than relying on execution time alone.

One of the challenges faced during development was understanding the fundamental difference between the two algorithms. Dijkstra's algorithm is computationally efficient but assumes non-negative edge weights, whereas Bellman-Ford is slower but supports negative edge weights and can detect negative cycles. Implementing these conditions correctly required careful consideration of the edge-relaxation process and termination conditions.

The experimental analysis helped me understand the practical meaning of asymptotic complexity. Although theoretical complexity provides an estimate of how algorithms behave as input size increases, actual execution time can also be affected by hardware, programming language, implementation choices, memory management, and graph structure. The experiment therefore helped connect theoretical algorithm analysis with practical performance evaluation.

This assignment also improved my understanding of graph representations, shortest-path algorithms, greedy techniques, complexity analysis, experimental evaluation, and performance measurement. I learned that choosing an algorithm should not depend only on which algorithm is theoretically faster. Instead, the characteristics of the problem must be considered. For a transportation network with non-negative edge weights, Dijkstra is generally the more appropriate choice because of its computational efficiency. Bellman-Ford becomes more useful when negative edge weights or negative-cycle detection are required.

The project is also connected to SDG 9 – Industry, Innovation and Infrastructure because efficient route planning can contribute to better transportation and logistics infrastructure. Optimised routes can potentially reduce transportation time, fuel consumption, operating costs, and unnecessary resource utilisation.

Overall, this assignment helped me achieve the mapped course outcomes by analysing algorithmic efficiency, applying graph-based techniques, evaluating alternative solutions, and creating an experimentally validated solution. It strengthened my ability to move from theoretical algorithm design to practical implementation and evidence-based algorithm selection.